



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

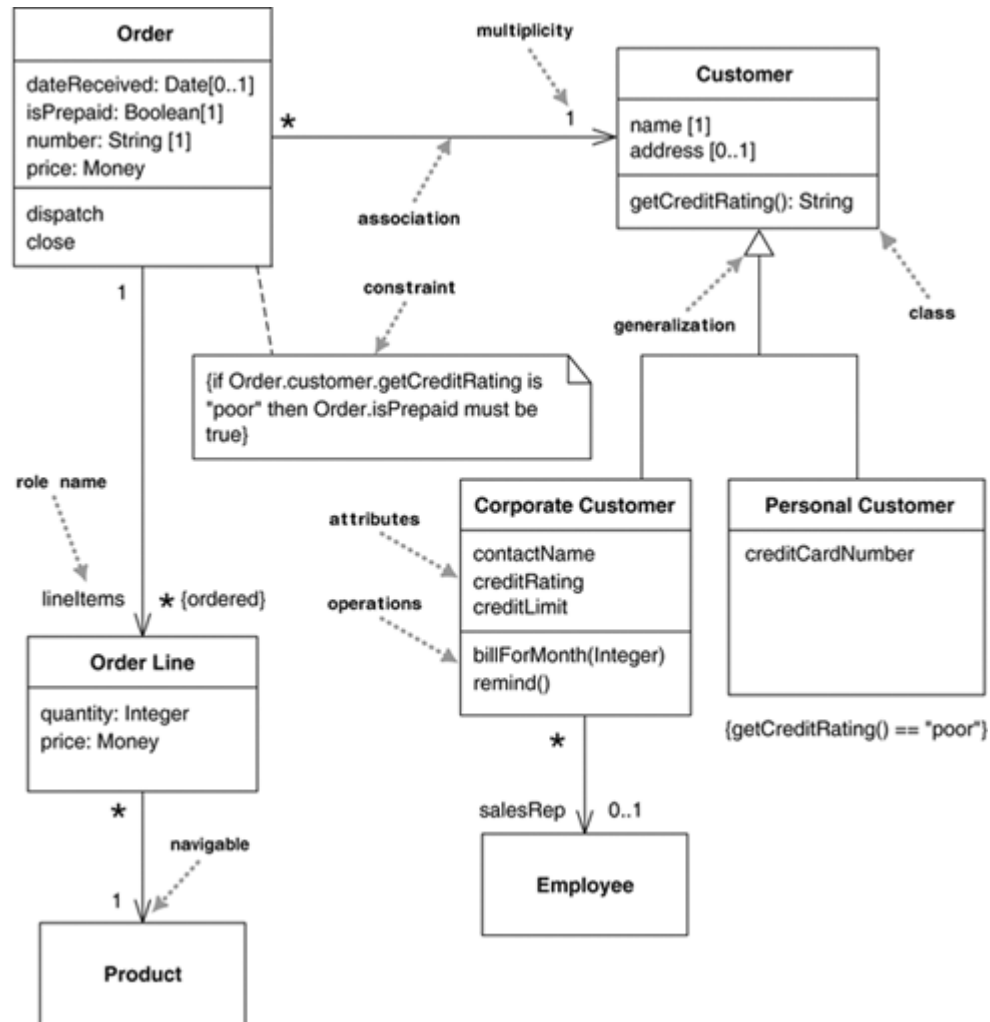
Class Diagrams: The Essentials

Francesco Guerra
francesco.guerra@unimore.it

Introduction

- ▶ A class diagram describes
 - ▶ the types of objects in the system,
 - ▶ the various kinds of static relationships that exist among them.
- ▶ Class diagrams also show
 - ▶ the properties and operations of a class
 - ▶ the constraints that apply to the way objects are connected.
- ▶ The UML uses the term feature as a general term that covers properties and operations of a class

A simple class diagram



Properties - Attributes

- ▶ Properties represent structural features of a class.
 - ▶ Properties as corresponding to fields in a class.
- ▶ Properties are a single concept, but they appear in two distinct notations:
 - ▶ attributes
 - ▶ associations
- ▶ The attribute describes a property as a line of text. The full form of an attribute is:
 - ▶ `visibility name: type multiplicity = default {property-string}`
 - ▶ `name: String [1] = "Untitled" {readOnly}`
- ▶ Only the name is necessary.
- ▶ This visibility marker indicates whether it is public (+), private (-), protected (# - protected elements on classes can be accessed by methods that are part of your class and also by methods that are declared on any class that inherits from your class) or package (~)
 - ▶ The name corresponds to the name of a field in a programming language.
 - ▶ The type indicates a restriction on what kind of object may be placed in the attribute.
 - ▶ The default value is the value for a newly created object.
 - ▶ The {property-string} allows you to indicate additional properties for the attribute.

Visibility - issues

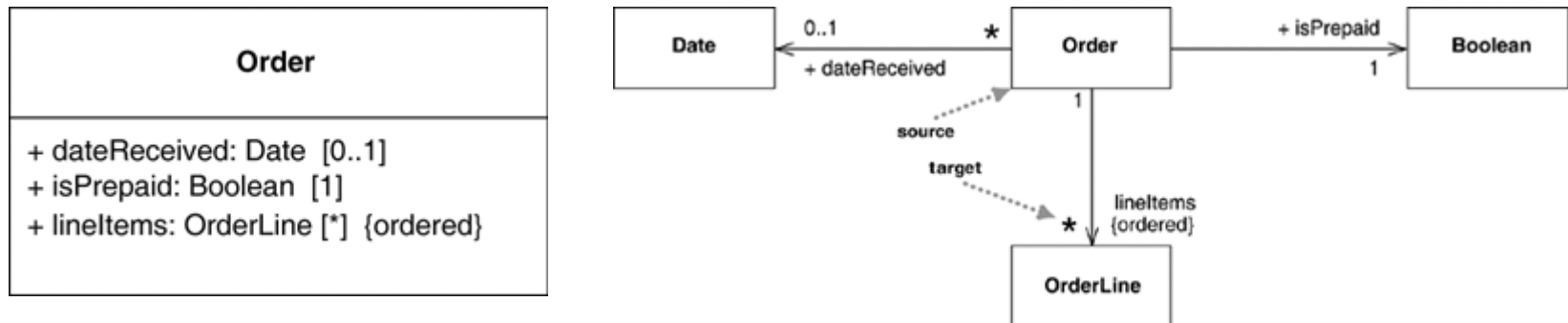
- ▶ To have public attributes or to not have public attributes?
 - ▶ Many object-oriented designers groan at the use of public attributes.
 - ▶ There is just as much potential for abuse.
 - ▶ It's usually best to avoid public attributes, but there are always exceptions to the rule.
 - ▶ One example can be to use a public attribute is when the attribute is a constant that may be used by a number of different classes (defining it as `readOnly`).
- ▶ Java confuses the matter a little by allowing access to protected parts of a class to any other class in the same package.
 - ▶ This is like combining the accessibility of protected and package visibility.

Properties

- Associations

- ▶ An association is a solid line between two classes,
 - ▶ directed from the source class to the target class.
 - ▶ The name of the property goes at the target end of the association, together with its multiplicity.
 - ▶ The target end of the association links to the class that is the type of the property.
- ▶ Associations can show multiplicities at both ends of the line.

Properties - Associations



- ▶ Why should you use one or the other?
 - ▶ Attributes for small things,
 - ▶ Associations for more significant classes

Multiplicity

- ▶ The multiplicity of a property is an indication of how many objects may fill the property.
 - ▶ 1 (An order must have exactly one customer.)
 - ▶ 0..1 (A corporate customer may or may not have a single sales rep.)
 - ▶ * (A customer need not place an Order and there is no upper limit to the number of Orders a Customer may place—zero or more orders.)
- ▶ Multiplicities are defined with a lower bound and an upper bound, e.g. 2..4 for players of Briscola.
- ▶ If the lower and upper bounds are the same, you can use one single number;
 - ▶ 1 is equivalent to 1..1.

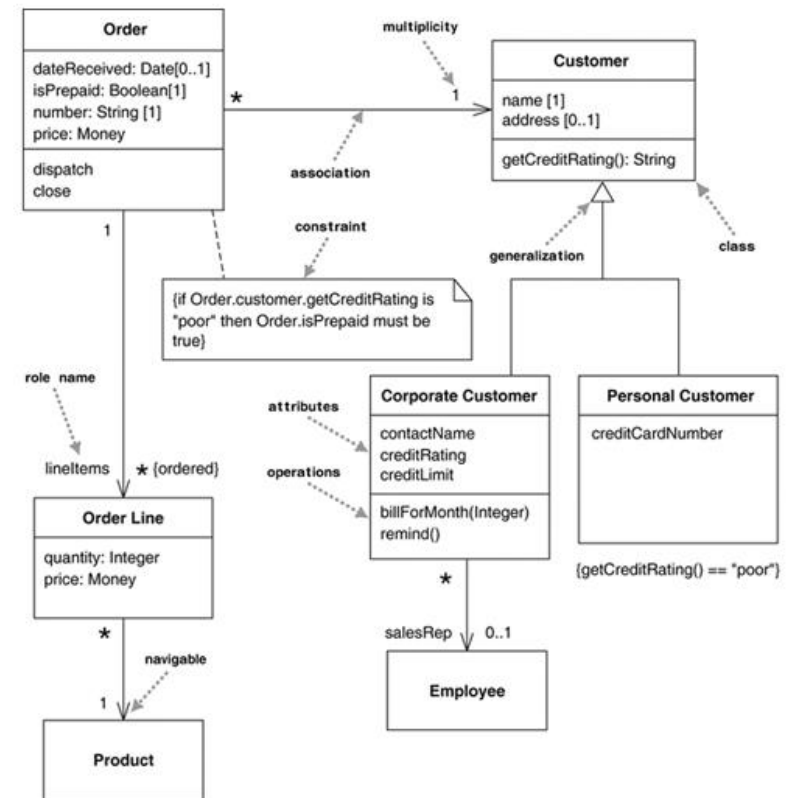
Multiplicity (2)

- ▶ There are various terms that refer to the multiplicity.
 - ▶ Optional implies a lower bound of 0.
 - ▶ Mandatory implies a lower bound of 1 or possibly more.
 - ▶ Single-valued implies an upper bound of 1.
 - ▶ Multivalued implies an upper bound of more than 1: usually *.
- ▶ By default, the elements in a multivalued multiplicity form a set,
 - ▶ If you ask a customer for its orders, they do not come back in any order.
 - ▶ If the ordering of the orders has meaning, you need to add {ordered} to the association end.
 - ▶ If you want to allow duplicates, add {nonunique}.
 - ▶ (If you want to explicitly show the default, you can use {unordered} and {unique}.)
 - ▶ You may also see collection-oriented names, such as {bag} for unordered, nonunique.
- ▶ The default multiplicity of an attribute is [1].
 - ▶ Although this is true in the meta-model, you can't assume that an attribute in a diagram that's missing a multiplicity has a value of [1], as the diagram may be suppressing the multiplicity information.

Programming Interpretation of Properties

- ▶ As with anything else in the UML, there's no one way to interpret properties in code.
- ▶ The most common software representation is that of a field or property of your programming language.

```
public class OrderLine...
    private int quantity;
    private float price;
    private Product product;
```



Programming Interpretation of Properties (2)

- ▶ Using private fields is a very implementation-focused interpretation of the diagram.
- ▶ A more interface-oriented interpretation might instead concentrate on the getting methods rather than the underlying data.

```
public class OrderLine...  
    private int quantity;  
    private Product product;  
    public int getQuantity() {  
        return quantity;  
    }  
    public void setQuantity(int quantity) {  
        this.quantity = quantity;  
    }  
    public Money getPrice() {  
        return product.getPrice().multiply(quantity);  
    }
```

there is no data field for
price;
it's a computed value.

Is this a problem for the
“client” of the class?

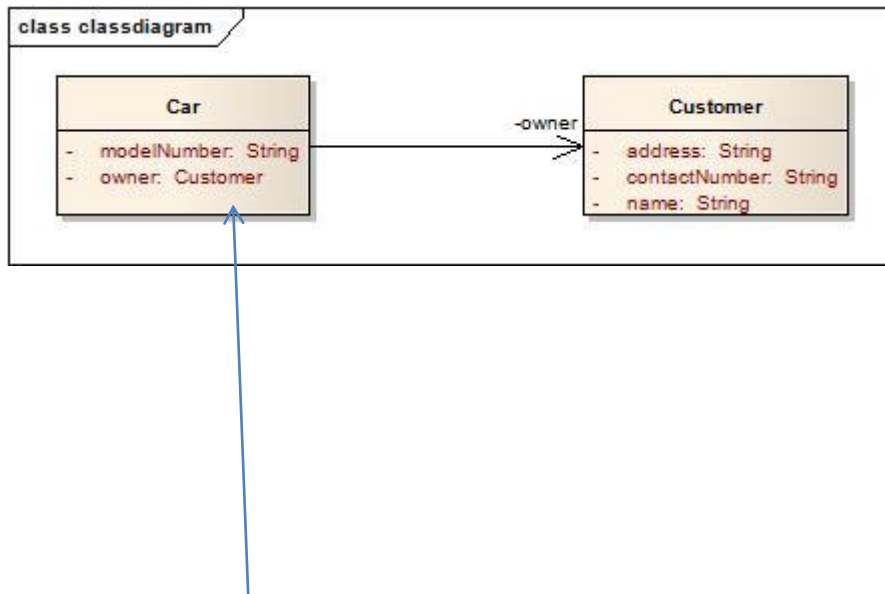
Programming Interpretation of Properties (3)

- ▶ If an attribute is multivalued, the data concerned is a collection.
 - ▶ An Order class would refer to a collection of Order Lines.
- ▶ Because in this case this multiplicity is ordered, that collection must be ordered.
- ▶ If the collection is unordered, thus be implemented with a set,
- ▶ Most people implement unordered attributes as lists as well.
- ▶ Some people use arrays, but the UML implies an unlimited upper bound.
 - ▶ Multivalued properties yield a different kind of interface to single-valued properties (in Java):

Programming Interpretation of Properties (4)

```
class Order {  
    private Set lineItems = new HashSet();  
  
    public Set getLineItems() {  
        return Collections.unmodifiableSet(lineItems);  
    }  
    public void addLineItem (OrderLine arg) {  
        lineItems.add (arg);  
    }  
    public void removeLineItem (OrderLine arg) {  
        lineItems.remove(arg);  
    }  
}
```

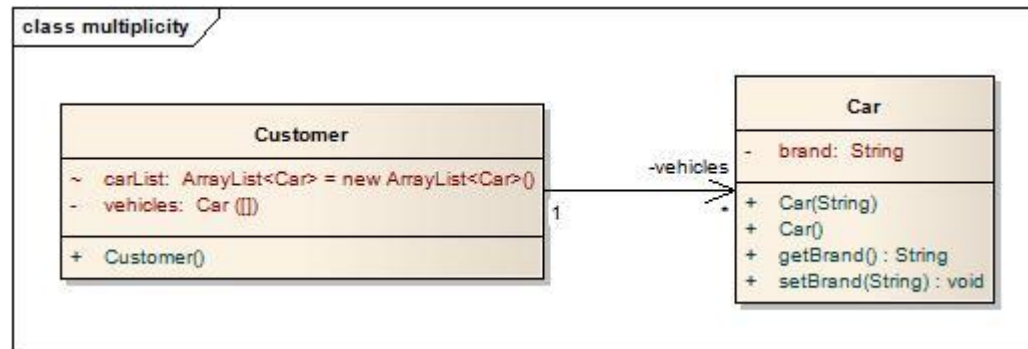
Example



```
public class Customer {
    private String name;
    private String address;
    private String contactNumber;
}
```

```
public class Car {
    private String modelNumber;
    private Customer owner;
}
```

Example



```

public class Car {
    private String brand;
    public Car(String brands){
        this.brand = brands;
    }
    public Car() {
    }
    public String getBrand() {
        return brand;
    }
    public void setBrand(String brand) {
        this.brand = brand;
    }
}
  
```

```

public class Customer {
    private Car[] vehicles;
    Protected ArrayList<Car> carList = new
    ArrayList<Car>();
    public Customer(){
    }
}
  
```

Bidirectional Associations



- ▶ A bidirectional association is a pair of properties that are linked together as inverses.
 - ▶ The Car class has property `owner:Person[1]`, and the Person class has a property `cars:Car[*]`.
- ▶ The inverse link between them implies that if you follow both properties, you should get back to a set that contains your starting point.

Bidirectional Associations



- ▶ As an alternative to labeling an association by a property, many people, particularly if they have a data-modeling background, like to label an association by using a verb phrase so that the relationship can be used in a sentence.
 - ▶ This is legal and you can add an arrow to the association to avoid ambiguity.
- ▶ Implementing a bidirectional association in a programming language is often a little tricky because you have to be sure that both properties are kept synchronized

Example



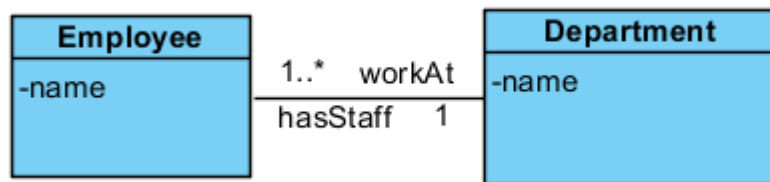
```
public class Customer {
    private String name;
    private String address;
    private String contactNumber;
    private Car car;
}
```

Is this enough?

```
public class Car {
    private String modelNumber;
    private Customer owner;
}
```

Is this correct?

Issues in implementing bi-directional associations



```

public class Employee {
    private String name;
    private Department dept;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Department getDept() {
        return dept;
    }
    public void setDept(Department
dept) {
        this.dept = dept;
    }
}
  
```

```

public class Department {
    private String name;
    private HashSet<Employee> staff;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public HashSet<Employee> getStaff() {
        return staff;
    }
    public void setStaff(HashSet<Employee>
staff) {
        this.staff = staff;
    }
}
  
```

Issues in implementing bi-directional associations (2)

- ▶ We have to guarantee that when we move an employee from one department to another, the software correctly update all the references
- ▶ Let us suppose that the employee "Francesco" is in the Economics department.
- ▶ If he moves in the Engineering department we have
 - ▶ To add Francesco in the Engineering department Staff
 - ▶ To remove Francesco from the Economics department Staff
- ▶ Is this enough?
 - ▶ No, we have to update the department associated to the Francesco object!

A possible implementation

```
public class Employee {
    private String name;
    private Department dept;
    ...
    public void setDept(Department dept)
    {
        dept.addEmployee(this);
    }
    public void updateDept(Department
    dept) {
        this.dept = dept;
    }
}
```

```
public class Department {
    private String name;
    private HashSet<Employee> staff = new
    HashSet<Employee>();
    public void setStaff(HashSet<Employee>
    staff) {
        this.staff = staff;
    }
    public void addEmployee(Employee employee) {
        if (employee.getDept() != null){
            employee.getDept().remove(employee);
        }
        staff.add(employee);
        employee.updateDept(this);
    }
    public void fire(Employee employee) {
        staff.remove(employee);
        employee.setDept(null);
    }
    public void remove(Employee employee) {
        staff.remove(employee);
    }
}
```

A possible implementation (2)

```
public class TestDrive {  
    public static void main(String[] args) {  
        Department economics;  
        Department engineering;  
        Employee francesco;  
        economics = new Department();  
        economics.setName("Economics");  
        engineering = new Department();  
        engineering.setName("Engineering");  
        francesco = new Employee();  
        francesco.setName("Francesco");  
        economics.addEmployee(francesco);  
        System.out.println(francesco.getName() + " works at " + francesco.getDept().getName());  
        System.out.println(economics.getName() + " has " + economics.getStaff().toString());  
        System.out.println(engineering.getName() + " has " + engineering.getStaff().toString());  
        francesco.setDept(engineering);  
        System.out.println(francesco.getName() + " works at " + francesco.getDept().getName());  
        System.out.println(economics.getName() + " has " + economics.getStaff().toString());  
        System.out.println(engineering.getName() + " has " + engineering.getStaff().toString());  
    }  
}
```

Francesco works at Economics
Economics has [Employee@2a139a55]
Engineering has []
Francesco works at Engineering
Economics has []
Engineering has [Employee@2a139a55]

Operations

- ▶ Operations are the actions that a class knows to carry out.
 - ▶ Operations most obviously correspond to the methods on a class.
 - ▶ Operations that simply manipulate properties are not shown
- ▶ The full UML syntax is: visibility name (parameter-list) : return-type {property-string}
 - ▶ This visibility marker is public (+) or private (-);
 - ▶ The name is a string.
 - ▶ The parameter-list is the list of parameters for the operation.
 - ▶ The return-type is the type of the returned value, if there is one.
 - ▶ The property-string indicates property values that apply to the given operation (e.g., abstract).
- ▶ The parameters in the parameter list are notated in a similar way to attributes. The form is: direction name: type = default value
 - ▶ The name, type, and default value are the same as for attributes.
 - ▶ The direction indicates whether the parameter is input (in), output (out) or both (inout). If no direction is shown, it's assumed to be in.
- ▶ An example operation on account might be: + balanceOn (date: Date) : Money

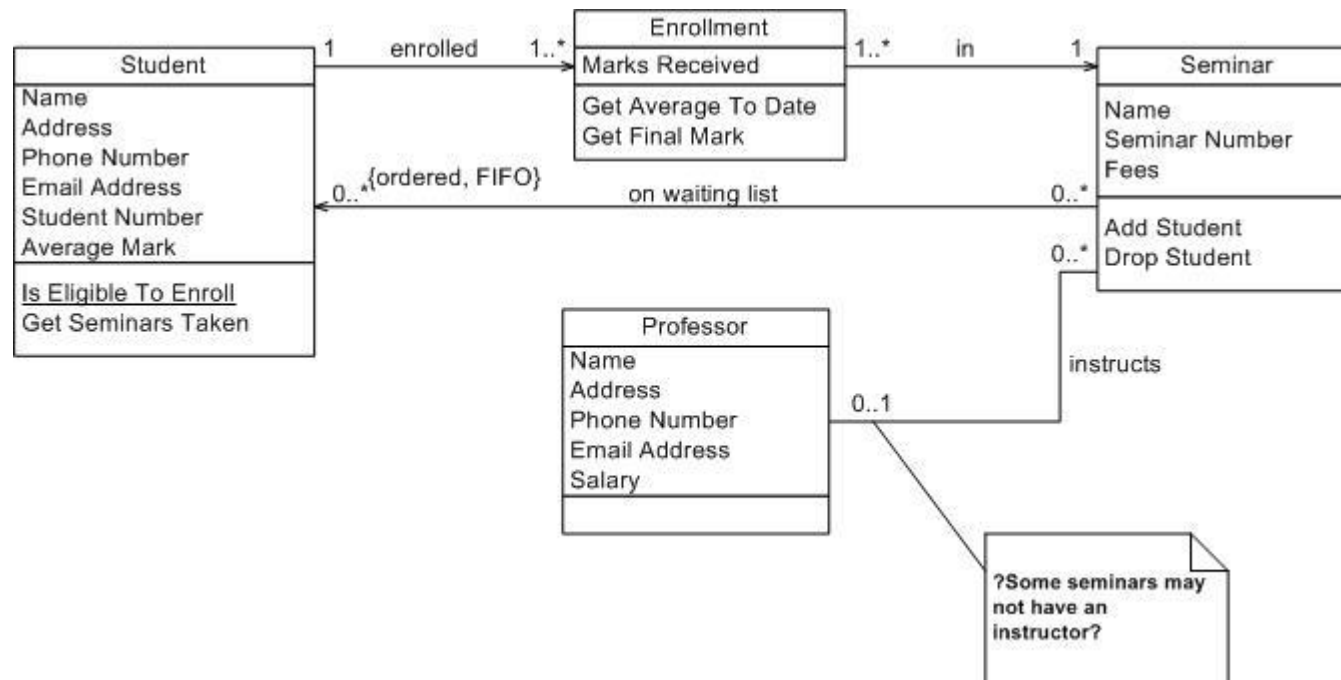
Operations (2)

- ▶ With conceptual models, you shouldn't use operations to specify the interface of a class.
 - ▶ Query as an operation that gets a value from a class without changing the state, i.e., without side effects. You can mark such an operation with the property string {query}.
- ▶ Modifiers, also called commands, i.e. operations that change the state of the system
 - ▶ The difference between query and modifiers is whether they change the observable state [Meyer].
 - ▶ The observable state is what can be perceived from the outside. An operation that updates a cache would alter the internal state but would have no effect that's observable from the outside.
- ▶ You can change the order of execution of queries and not change the system behavior.
- ▶ A common convention is to write operations so that modifiers do not return a value;
 - ▶ [Meyer] refers to this as the **Command-Query separation principle**.

Operations (3)

- ▶ Operations vs Methods
 - ▶ An operation is something that is invoked on an object—the procedure declaration—whereas a method is the body of a procedure.
 - ▶ They are different when you have polymorphism.
 - ▶ If you have a supertype with three subtypes, each of which overrides the supertype's `getPrice` operation, you have one operation and four methods that implement it.
- ▶ People usually use the terms operation and method interchangeably

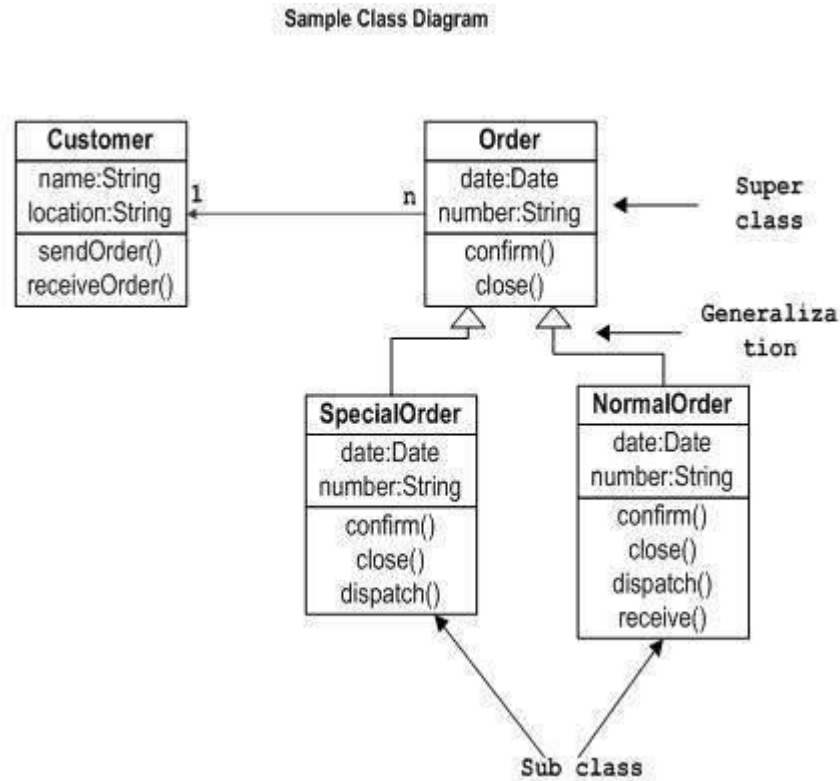
Examples



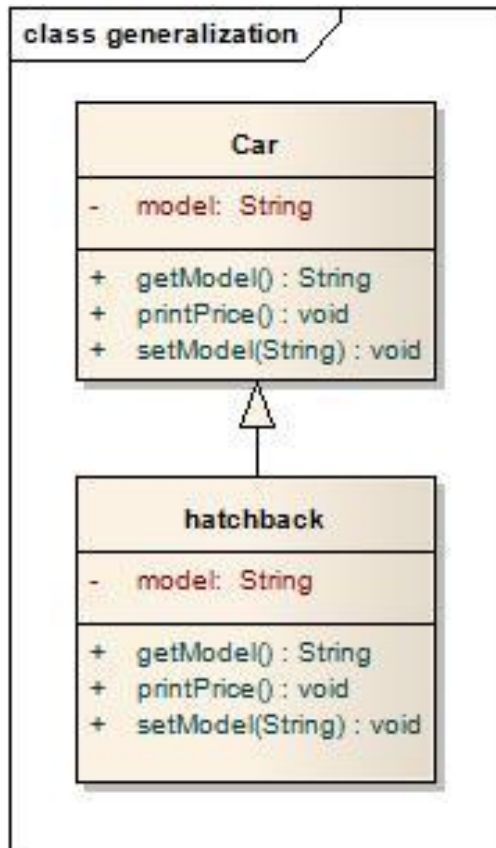
Generalization

- ▶ With a software perspective, the obvious interpretation is inheritance:
 - ▶ The Corporate Customer is a subclass of Customer.
- ▶ In mainstream OO languages, the subclass inherits all the features of the superclass and may override any superclass methods.
- ▶ An important principle of using inheritance effectively is substitutability.
 - ▶ I should be able to substitute a Corporate Customer within any code that requires a Customer, and everything should work fine.
 - ▶ if I write code assuming I have a Customer, I can freely use any subtype of Customer.

Example



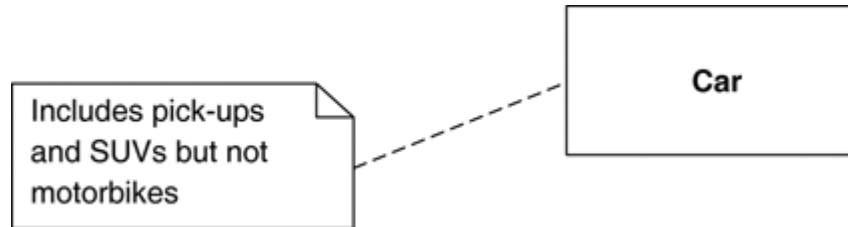
Example in java



```
public class Car {
    private String model;
    public void printPrice() {
    }
    public String getModel() {
        return model;
    }
    public void setModel(String model) {
        this.model = model;
    }
}

public class hatchback extends Car {
    private String model;
    public void printPrice() {
        System.out.println("Hatchback Price");
    }
    public String getModel() {
        return model;
    }
    public void setModel(String model) {
        this.model = model;
    }
}
```

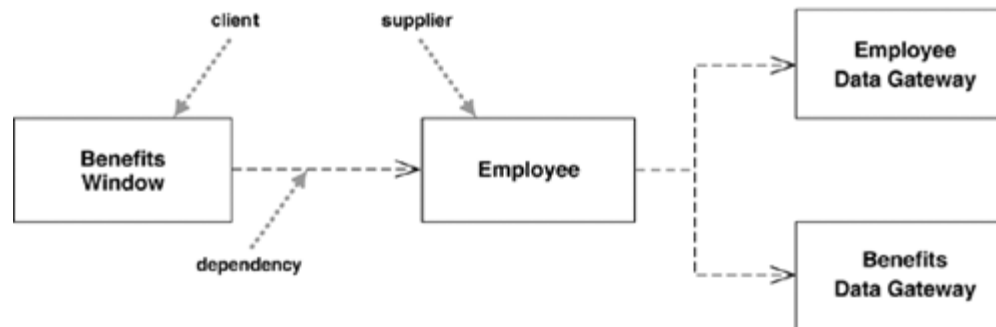
Notes and Comments



- ▶ Notes are comments in the diagrams.
 - ▶ Notes can stand on their own, or they can be linked with a dashed line to the elements they are commenting

Dependency

- ▶ A dependency exists between two elements if changes to the definition of one element (the supplier) may cause changes to the other (the client).



Dependency

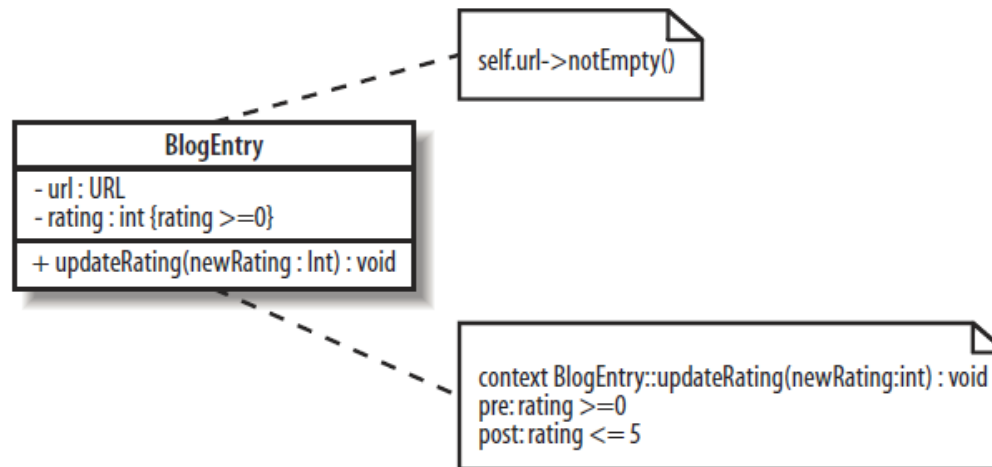
Keyword	Meaning
«call»	The source calls an operation in the target.
«create»	The source creates instances of the target.
«derive»	The source is derived from the target.
«instantiate»	The source is an instance of the target. (Note that if the source is a class, the class itself is an instance of the class class; that is, the target class is a metaclass).
«permit»	The target allows the source to access the target's private features.
«realize»	The source is an implementation of a specification or interface defined by the target.
«refine»	Refinement indicates a relationship between different semantic levels; for example, the source might be a design class and the target the corresponding analysis class.
«substitute»	The source is substitutable for the target .
«trace»	Used to track such things as requirements to classes or how changes in one model link to changes elsewhere.
«use»	The source requires the target for its implementation.



Constraint Rules

- ▶ Much of what you are doing in drawing a class diagram is indicating constraints.
- ▶ The basic constructs of association, attribute, and generalization do much to specify important constraints, but they cannot indicate every constraint.
- ▶ These constraints still need to be captured; the class diagram is a good place to do that.
- ▶ The UML allows you to use anything to describe constraints. The only rule is that you put them inside braces ({}).
 - ▶ You can use natural language, a programming language, or the UML's formal Object Constraint Language (OCL) [Warmer and Kleppe], which is based on predicate calculus.
 - ▶ Optionally, you can name a constraint by putting the name first, followed by a colon; for example, {disallow incest: husband and wife must not be siblings}.

Constraint Rules OCL (2)



- ▶ There are 3 types of constraint using OCL:
 - ▶ *Invariants*
 - ▶ An *invariant* is a constraint that must always be true; otherwise the system is in an invalid state. Invariants are defined on class attributes.
 - ▶ *Preconditions*
 - ▶ A *precondition* is a constraint that is defined on a method and is checked before the method executes. Preconditions are frequently used to validate input parameters to a method.
 - ▶ *Postconditions*
 - ▶ A *postcondition* is also defined on a method and is checked after the method executes. Postconditions are frequently used to describe how values were changed by a method.

When to Use Class Diagrams

- ▶ Class diagrams are the backbone of the UML
- ▶ The trouble with class diagrams is that they are so rich, they can be overwhelming to use. Here are a few tips.
 - ▶ Don't try to use all the notations available to you. Start with the simple stuff: classes, associations, attributes, generalization, and constraints. Introduce other notations when you need them.
 - ▶ Conceptual class diagrams are very useful in exploring the language of a business. For this to work, you have to work hard on keeping software out of the discussion and keeping the notation very simple.
 - ▶ Don't draw models for everything; instead, concentrate on the key areas. It is better to have a few diagrams that you use and keep up to date than to have many forgotten, obsolete models.
- ▶ The biggest danger with class diagrams is that you can focus exclusively on structure and ignore behavior. Therefore, when drawing class diagrams to understand software, always do them in conjunction with some form of behavioral technique. If you're going well, you'll find yourself swapping between the techniques frequently.

Esercizio

- ▶ Una banca ha numerosi clienti.
- ▶ Ogni cliente può avere uno o più conti correnti e li usa per depositare e prelevare soldi, per acquistare titoli di stato e per accendere mutui.
- ▶ Un cliente è identificato da almeno un nome e un domicilio, un conto corrente ha un numero e un saldo, un titolo di stato è definito da un nome, un costo e una scadenza e un mutuo deve avere un valore, una scadenza e una periodicità.

Esercizio

- ▶ Un'azienda è divisa in sedi, dove lavorano sia dipendenti tecnici sia amministrativi. Ogni lavoratore percepisce uno stipendio. Il lavoratore può chiedere di cambiare sede, venire licenziato o dare le dimissioni.
- ▶ Gli amministrativi possono avere avanzamenti di carriera. Per i tecnici questo significa un cambiamento di mansione. Le due tipologie di lavoratori adottano modalità diverse per il calcolo degli stipendi
- ▶ Ogni sede ha un budget proprio e provvede agli stipendi dei propri lavoratori in modo autonomo. L'azienda vuole sapere il numero totale dei dipendenti.



References

- ▶ Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language , Pearson Addison Wesley
- ▶ <http://www.edc4it.com/blog/java/implementing-a-bi-directional-association-in-java.html>
- ▶ Russ Miles, Kim Hamilton: Learning UML 2.0 A Pragmatic Introduction to UML, O'Reilly Media, 2006
- ▶ https://vaughnvernon.co/?page_id=31

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.